Verifying the RISC-V IOMMU: An Open Verification Environment and a Fault-Injection Evaluation

Maulik Timbadiya, Independent Researcher

Abstract—The RISC-V IOMMU specification reached version 1.0 in 2024 and the first open-source RTL implementations have appeared, but there is no published verification methodology for them, and the leading open implementation ships with no testbench at all. We report an open, licence-free verification environment for that IP and an evaluation of what it actually catches. The environment drives all four AXI interfaces of the device, serves its implicit page-table, command and fault-queue accesses from a memory model, and checks every translation against an independent software walker; it is 1,144 lines of SystemVerilog, builds in 2.4 s and runs 56 checks in 5 ms using Verilator alone. We evaluate it by injecting eleven semantically meaningful IOMMU faults into the RTL. An early suite detected four of nine; the survivors identified concrete blind spots — page-table permission bits, reserved-bit encodings and second-stage translation — and the tests written to close them brought detection to eleven of eleven. We report two findings about the IP: it cannot be linted as published because a dependency is absent, and, with the fault queue disabled, eight faulting device accesses leave the IOMMU unresponsive and a previously valid translation fails. We also report a methodology result we think is more broadly useful: during development a protocol-violating bus driver produced three plausible, reproducible and entirely false IP defects, which argues for protocol checking on the stimulus side of a testbench and not only on the device under test.

Index Terms—IOMMU, RISC-V, functional verification, fault injection, mutation testing, address translation, SystemVerilog, open-source hardware.

I. Introduction

An IOMMU stands between devices and memory. It translates the addresses a device emits, enforces the permissions of the domain that owns the device, and is the mechanism by which an operating system or hypervisor confines a DMA-capable peripheral. Because it is a confinement mechanism, a defect in it is a containment failure: a device may reach memory it was never granted, or a stale cached translation may outlive the mapping it came from.

The RISC-V IOMMU specification reached version 1.0, and open RTL implementations have followed. The most complete of these is the IP published by Zero-Day Labs [1], which implements device-directory walks, two-stage Sv39/Sv39x4 translation, an IOTLB and directory caches, a command queue and a fault queue. Its own documentation states that the IP "is not formally verified" and is "very likely to have bugs." It ships no testbench: its makefile invokes Verilator in –lint-only mode.

That gap is the subject of this article. There is a substantial literature on verifying RISC-V cores—random instruction generators, reference-model co-simulation, coverage closure [2], [3]—and a separate literature on verifying address-translation ordering for CPU MMUs [4], [5]. We are not aware of published work on verifying a RISC-V IOMMU. This article does not propose a new verification theory. It reports what it takes to build a usable environment for this class of device, and—more importantly—measures how much of the device's behaviour that environment actually exercises, using fault injection rather than assertion counts or line coverage.

A. Contributions

1) An open verification environment for the RISC-V IOMMU that needs only Verilator, Yosys is not required, and no commercial licence. It drives the programming, device-request, completion and data-structure interfaces, and checks translations against an independent walker.
2) A fault-injection evaluation with eleven injected IOMMU defects, and the trajectory that matters: an early suite caught four of nine, the survivors named the blind spots, and the resulting tests caught eleven of eleven.
3) Two reproducible findings about the IP: it cannot be linted as published, and disabling the fault queue lets a faulting device render the IOMMU unresponsive.
4) A methodology result: a bus driver that violated the AXI handshake manufactured three convincing false defects in the device under test.

II. Background

A. What the device does

A device issues a transaction carrying a device identifier. The IOMMU uses that identifier to walk a device-directory table (DDT) in memory and obtain a device context, which names the translation structures for that device: a first-stage table (Sv39, the device's own virtual addresses), a second-stage table (Sv39x4, the guest-to-supervisor mapping) or neither, in which case the transaction passes through. The resulting translation is cached in an IOTLB, and the device context itself in a directory cache.

Because those caches are filled from memory the host also writes, software must be able to invalidate them.

The IOMMU exposes a command queue; software enqueues IOTINVAL.VMA to invalidate translations and IOFENCE.C to learn when previous commands have taken effect. Faults—an invalid device context, a missing page-table entry, a permission violation—are reported by appending a record to a fault queue and by aborting the offending transaction.

B. Why this is awkward to verify

Three properties make an IOMMU harder to drive than a core. First, its stimulus is not instructions but bus transactions on one port, with the response appearing on a different port: the translated transaction is issued by the IOMMU as a master, so the quantity under test—the translated address—is observed on an interface the testbench must implement as a slave. Second, most of its behaviour depends on data structures in memory, so a testbench must construct valid device-directory and page-table images before any stimulus is meaningful. Third, its caches make behaviour history-dependent: the same transaction is correct or incorrect depending on what was invalidated before it.

III. Related Work

Verification of RISC-V cores is well served. riscv-dv [2] generates constrained-random instruction streams and core-v-verif provides UVM environments with reference-model co-simulation [3]; both target cores, require a commercial simulator for their UVM components, and do not address an IOMMU.

Address translation has been studied from the ordering side. COATCheck models memory ordering at the hardware–OS interface including translation events [4], and TransForm synthesises litmus tests for such "transistency" models [5]. RTLCheck verifies that RTL satisfies the microarchitectural axioms a consistency proof assumes [7]. These concern a CPU's MMU and its interaction with software; none targets a device-side IOMMU, and none constructs a functional verification environment for one. The invalidation-ordering subset of that gap is the subject of a companion article [6]; here we are concerned with the environment and with what fault injection says about the suite as a whole.

Fault injection as a way of measuring a test suite is long established in software as mutation testing, and has been applied to hardware description languages to assess verification quality. We use it here not to claim novelty in the technique but because it answers the question that matters for a new environment: not how many lines were executed, but which defects would survive.

IV. Device Under Test

We target the Zero-Day Labs RISC-V IOMMU IP [1] at commit 18ad30a: 75 SystemVerilog files, 25,778 lines, Apache-2.0 and Solderpad licensed. It implements one-, two- and three-level device directories, Sv39 first-stage and Sv39x4 second-stage translation with superpages, a fully-associative IOTLB, device- and process-directory caches, the command and fault queues, wired and message-signalled interrupts, and performance counters. It does not implement the PCIe ATS command set.

The IP exposes four AXI4 interfaces: a slave port on which devices issue transactions, a master port on which translated transactions leave, a master port for its own implicit accesses (directory walks, page-table walks, queue accesses), and a programming slave port.

A. It cannot be linted as published

Our first observation is prosaic and immediate. The register map includes common_cells/assertions.svh, which the repository does not vendor, so the IP's own lint flow fails on a clean checkout. Supplying the upstream lowRISC header does not fix it: that header defines ASSERT_RPT with two arguments where this IP's RTL uses one, and the build fails on a macro arity error. We resolved it by forwarding the missing path to the IP's own bundled assertions.svh, which keeps one self-consistent set of macros. With that one file supplied, the IP lints clean and elaborates under Verilator.

V. The Verification Environment

Fig. 1 shows the structure. Four components surround the device.

A programming driver performs register reads and writes over AXI4. It supports both 8-byte and 4-byte accesses, which matters: several IOMMU registers are 32 bits wide and the ddtp register commits only when both of its 32-bit halves have been written, so an environment that can only issue 8-byte accesses cannot program the device correctly, and one that masks sub-word addresses to the 8-byte boundary will write the neighbouring register.

A data-structure memory model serves the IOMMU's implicit accesses as an AXI4 slave over a 512 KiB sparse image. Every structure the device reads—the device directory, both levels of page tables, the command queue—and every structure it writes—fault records, fence completion words—lives in this image, which the testbench also builds and inspects directly.

A device request driver issues reads and writes on the translation request port, carrying a device identifier in the AXI user-defined fields.

A completion checker implements the master port as a slave, recording the translated address of every transaction the IOMMU forwards.

An independent walker re-implements the Sv39 walk in the testbench, reading the same memory image, and yields the address each transaction should produce. Checks compare the observed translated address against it rather than against a constant, so a test remains valid when the page tables change.

A. Constructing the data structures

Nothing in the device is reachable until memory contains structures it accepts, and the encodings are unforgiv-

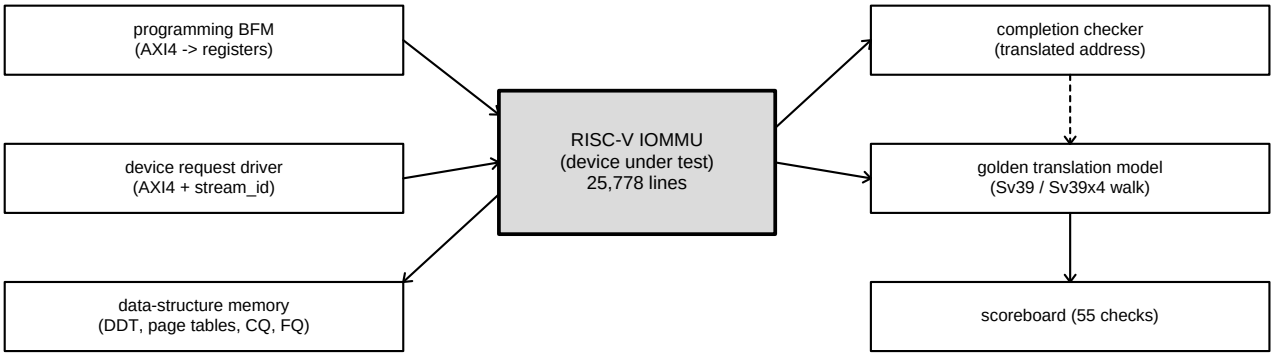


Fig. 1. Environment structure. The translated address—the quantity under test—is observed on the completion port, which the testbench implements as a slave, and compared against an independent walk of the same memory image.

ing. A base-format device context is four 64-bit words—translation control, second-stage pointer, translation attributes, first-stage pointer—placed at DDT_BASE + device identifier × 32 in one-level mode. Only bit 0 of the first word, the valid bit, must be set for a minimal context; the two pointer words select translation mode in their top four bits, so leaving both zero yields a valid device whose transactions pass through untranslated. That configuration is the smallest end-to-end test in the suite and the one we brought up first: it exercises register programming, the directory walk and the completion path while removing page tables from the problem.

First-stage page-table entries follow the Sv39 format: the physical page number occupies bits 53:10 and the permission bits occupy the low byte. Our builder writes three levels, with non-leaf entries carrying only the valid bit and the leaf carrying valid, read, write, user, accessed and dirty. The second stage uses Sv39x4, whose root table is 16 KiB and is indexed by eleven bits of the guest address rather than nine; getting that index width wrong produces a walk that reads a plausible but wrong table entry, which is why the second-stage test checks the number of implicit accesses as well as the final address.

B. Command queue mechanics

The command queue is a power-of-two ring of 16-byte entries whose base and size are programmed together, with software advancing a tail index and the IOMMU advancing a head index. Two details matter for a testbench. The tail index must wrap: we initially incremented it without wrapping, which wrote commands past the end of the queue and left the device with a tail index outside the ring, so invalidation silently stopped working while every earlier test still passed. And the queue's registers are 32 bits wide and adjacent, so an 8-byte write aimed at the tail also writes the head.

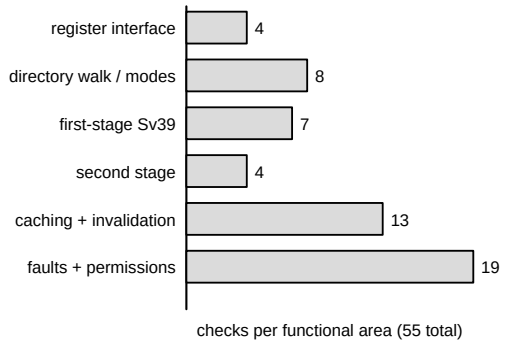


Fig. 2. Distribution of the 55 functional checks (a 56th establishes a precondition).

IOFENCE.C carries an address and a data word; when the address-valid bit is set the IOMMU writes that data to that address once preceding commands have taken effect. Because the address field is expressed in units of four bytes, the testbench polls its own memory image for the data word and uses its appearance as the ordering reference point for the invalidation tests.

C. Observing responses correctly

Both drivers determine completion by counting channel handshakes rather than by sampling a valid level. This is not fastidiousness. An early version sampled levels, and after a faulting transaction it mistook a stale response for its own, producing tests that reported a successful read with no corresponding downstream transaction—an impossible combination that cost real time to diagnose. Section VIII-C returns to this.

VI. Test Suite

The suite is 56 checks grouped as in Fig. 2.

TABLE I
Test areas, with the number of checks in each

Area	#	Representative check
Register interface	4	4-byte write takes effect
Directory, modes	8	context read from right address
First-stage Sv39	7	32 random maps match walker
Second stage Sv39x4	4	GPA maps to expected SPA
Caching, invalidation	13	no stale entry after fence
Faults, permissions	19	dirty bit clear faults a write
Total	55	(plus 1 precondition check)

Register interface. Capability reporting; 4-byte and 8-byte register access; that a sub-word access does not disturb the following write.

Directory walk and modes. ddtp programming including the two-half commit protocol and the busy handshake; Bare mode passing transactions through untranslated and performing no directory walk; one-level directory mode fetching the device context from the expected address (DDT_BASE + device identifier × 32); and that ddtp can still be reprogrammed after device traffic.

First-stage translation. A mapped page translating to the address the independent walker predicts, with the walk visiting the directory and all three page-table levels; a second access to the same page served from the IOTLB with no walk; and 32 randomly chosen mappings accessed at random offsets, each compared against the walker.

Second stage. A device configured with first stage Bare and second stage Sv39x4, so that the address it emits is a guest physical address; translation to the expected supervisor address; and that a G-stage entry with the user bit clear faults, since device accesses are user-mode.

Caching and invalidation. That a cached translation survives a page-table change until invalidated; that IOTINVAL.VMA forces a re-walk and exposes the new mapping; that an address-scoped invalidation refreshes the named page and leaves others cached; that an invalidate-all refreshes every cached page; that IOFENCE.C writes its completion word; and that no stale translation is observed after that word becomes visible, including with 24 device accesses in flight throughout.

Faults and permissions. An invalid device context, an unmapped address and ddtp=Off each abort the transaction and, importantly, produce no transaction on the completion port; faulting reads and writes both receive error responses; faults are recorded in the fault queue; and page-table entries with the accessed bit clear, the dirty bit clear on a write, the user bit clear, a reserved bit set, or the valid bit clear each fault.

Table I lists the areas. The whole suite builds in 2.4 s and runs in 5 ms, which matters more than it sounds: a suite that reruns in seconds is a suite a designer will actually run, and it is what makes the fault-injection campaign in the next section cheap enough to repeat after every change.

TABLE II
Injected faults and whether the suite detects them

	Injected defect	Detected
M1	first-stage PTE valid bit ignored	yes
M2	PTE accessed bit ignored	yes
M3	PTE dirty bit ignored on stores	yes
M4	PTE user bit ignored (second stage)	yes
M5	PTE reserved-bit check removed	yes
M6	ddtp=Off no longer blocks traffic	yes
M7	Bare decoded as a normal mode	yes†
M8	first-stage enable inverted	yes
M9	IOTLB invalidation does not clear entries	yes
M10	command-queue head never advances	yes
M11	invalidate-all clears only the first entry	yes

†detected as a hang: the suite fails to complete.

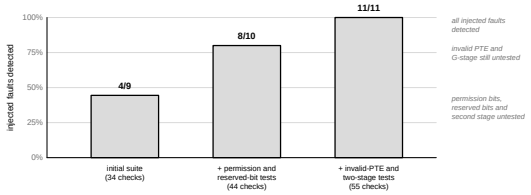


Fig. 3. Detection as the suite grew. The surviving faults named the blind spots; the tests written to close them are what raised detection.

VII. Fault-Injection Evaluation

Each fault is one edit to a private copy of the RTL, applied by script; the suite is rebuilt and rerun, and a fault counts as detected if any check fails. The faults are chosen to be semantically meaningful rather than syntactic: each corresponds to an omission a designer could plausibly make.

Table II gives the final result, eleven of eleven. The trajectory in Fig. 3 is the more useful result. The first suite of 34 checks detected four of nine usable faults. The survivors were not random: every page-table permission fault survived, because our tests always constructed entries with the accessed, dirty and user bits set; the reserved-bit fault survived because we never set reserved bits; and the second-stage fault survived because every test left the second stage in Bare mode. Writing tests for those five behaviours raised detection to eleven of eleven, and added the two-stage translation tests the environment had been missing entirely.

This is the argument for evaluating an environment this way. All 34 original checks passed on the unmodified RTL, and would have passed on an RTL missing every page-table permission check.

VIII. Findings

A. A faulting device can render the IOMMU unresponsive

With the fault queue disabled—fqcsr.fqen=0—we issue eight transactions that fault, then a transaction that previously succeeded. The precondition is checked first: immediately before the experiment, the valid translation is observed to work. Of the eight faulting accesses, four

receive a response and four receive none. The subsequent valid transaction then fails.

We report this as observed rather than as a specification violation, because the specification's requirements for fault handling when the queue is disabled are weaker than for the enabled case, and we did not isolate the internal cause. Its practical significance does not depend on that adjudication: software that has not yet enabled the fault queue—during early boot, or after an error—exposes a path by which a misbehaving or malicious device can stop the IOMMU servicing transactions at all.

B. Programming sequences are easy to get wrong

Two behaviours cost us time and would cost an integrator the same. The ddtp register commits only when both 32-bit halves have been written and no transaction is in flight, so a single 8-byte write works only because the bus bridge decomposes it into two 32-bit writes. And the fault queue must be enabled before faults are provoked, per the finding above. Both are visible in the RTL; neither is obvious from the register description.

C. A protocol-violating testbench manufactures defects

During development we recorded three defects in the IP, each reproducible, each with a plausible mechanism: 4-byte register writes were silently dropped; ddtp could not be reprogrammed once device traffic had occurred; and a sub-word access corrupted the following register write.

All three were false. Two independent faults in our own driver produced them. The driver deasserted w_valid one cycle late and therefore transferred two write-data beats for a single-beat burst, leaving a spare beat queued in the device's bus bridge so that every subsequent write used the previous transaction's data. Separately, it masked sub-word addresses to the 8-byte boundary, so a 4-byte write to a register at offset 0x24 was issued at 0x20 and wrote the neighbouring register. We found them by monitoring both sides of the device's internal bridge simultaneously and observing correct data entering and stale data leaving, with the write-data monitor firing twice per transaction.

The general point is that the stimulus side of a testbench is unverified code driving a protocol, and when it violates that protocol the device responds strangely in ways that are easy to attribute to the device. Every published "bug" in a new environment should be treated as a hypothesis about the testbench until the testbench has been cleared. Protocol checkers are routinely bound to the device under test; this experience suggests binding them to the drivers as well.

IX. Discussion

A. What this would need in a production flow

The environment here is deliberately small and licence-free, which bounds it. A production flow would keep three things and replace two. It would keep the independent walker, because checking a translation against a constant is worthless once page tables change during a test; it would keep the completion port as the observation point, because that is where the quantity under test appears; and it would keep fault injection as the measure of the suite, because no coverage metric would have told us the permission bits were untested.

It would replace the hand-written drivers with verified bus agents—the argument of Section VIII-C is precisely that hand-written drivers are a liability—and it would replace the fixed test list with constrained-random generation over the configuration space: directory depth, page size including superpages, process contexts, and the interrupt and MSI translation modes this work does not touch. Functional coverage would then be defined over that space, with fault injection retained as the check that the coverage model is not self-satisfied.

B. Generality

Nothing in the structure is specific to RISC-V. An Arm SMMU presents the same shape of problem: a stream identifier indexing a table of contexts, two translation stages, a translation cache, an invalidation command queue with a completion primitive, and an event queue. The four-component structure, the independent walker, and the fault-injection evaluation transfer directly; only the encodings change.

X. Threats to Validity

The evaluation measures detection of faults we chose; a different fault set would give a different score, and eleven of eleven should be read as "the suite covers these eleven behaviours," not as a general adequacy claim. The environment exercises one device-directory mode, one process-context configuration, 4 KiB pages and a 64-bit data bus; superpages, three-level directories, process contexts, message-signalled interrupt translation, performance counters and the debug interface are not exercised at all. The IP does not implement ATS, so no ATS behaviour is tested. All results come from one simulator; we did not repeat them on a commercial simulator. Finally, the fault-queue finding is reported as observed behaviour and its specification status is not settled here.

XI. Conclusion

An open RISC-V IOMMU implementation of 25,778 lines shipped without a testbench, and without one file needed to lint it. We built an environment that drives it end to end in 1,144 lines using only free tools, and then measured that environment by injecting defects rather than by counting coverage. The measurement was worth more than the environment: a suite that passed every check on the unmodified design would have missed every page-table permission fault, and only fault injection said so. The same discipline applied to our own testbench would have saved more time still, since three of the defects

we initially attributed to the device were produced by a driver that did not obey the bus protocol.

Artifact Availability

The environment, the fault-injection harness, the figure scripts and the raw results are released under a permissive licence. A single command builds the environment and runs the suite; a second reproduces the fault-injection table.

References

[1] Zero-Day Labs, “RISC-V IOMMU IP,” commit 18ad30a. [Online]. Available: https://github.com/zero-day-labs/riscv-iommu
[2] CHIPS Alliance, “riscv-dv: random instruction generator for RISC-V processor verification.” [Online]. Available: https://github.com/chipsalliance/riscv-dv
[3] OpenHW Group, “core-v-verif.” [Online]. Available: https://github.com/openhwgroup/core-v-verif
[4] D. Lustig, G. Sethi, M. Martonosi, and A. Bhattacharjee, “COATCheck: Verifying memory ordering at the hardware–OS interface,” in ASPLOS, 2016.
[5] N. Manerkar, D. Lustig, M. Martonosi, and A. Bhattacharjee, “TransForm: Formally specifying transistency models and synthesizing enhanced litmus tests,” in ISCA, 2018.
[6] M. Timbadiya, “Device-side invalidation ordering: testing IOMMU translation caches at RTL,” companion article.
[7] Y. A. Manerkar, D. Lustig, M. Martonosi, and M. Pellauer, “RTLCheck: Verifying the memory consistency of RTL designs,” in MICRO, 2017.
[8] W. Snyder, “Verilator.” [Online]. Available: https://verilator.org